PROJECT FINAL PAPER- CPRE 5480

Team One: Scott Russell, John Beuter, and Nisha Raj

Project Platform:

For this project, we utilized the Contiki-NG operating system in conjunction with the Cooja network simulator to study the behavior of low-power wireless networks. This platform was selected because it is well suited for experimenting with constrained Internet of Things (IoT) devices and provides strong support for time-sensitive and low-power networking concepts central to cyber-physical systems (CPS).

Contiki-NG is a lightweight, open-source operating system designed for embedded devices with limited memory, processing capability, and energy availability. It includes a full IPv6-based networking stack tailored for low-power environments through 6LoWPAN, along with built-in support for the Routing Protocol for Low-Power and Lossy Networks (RPL). At the link layer, Contiki-NG supports IEEE 802.15.4 MAC protocols, including the default CSMA/CA scheme as well as Time Slotted Channel Hopping (TSCH). These capabilities allowed us to examine how different MAC-layer approaches affect reliability and timing behavior without needing to implement custom protocol logic from scratch.

From a network stack design perspective, Contiki-NG implements a layered architecture optimized for low-power IoT systems. IEEE 802.15.4 provides low-rate wireless communication

suitable for devices with resource limitations and operates at the physical and link layers. The MAC layer was configured to use either CSMA/CA or TSCH, enabling direct comparison between contention-based and time-synchronized communication. Above the MAC layer, IPv6 networking is supported through 6LoWPAN, which enables efficient communication over low-power links. At the network layer, RPL maintains routing across the three topologies we evaluated: star, mesh, and tree.

At the transport and application layers, UDP was selected due to its lightweight, connectionless design, which minimizes protocol overhead and allows performance metrics to be analyzed without additional complexity present if we utilized TCP. Because Cooja emulates Contiki-NG motes at the system level, changes to the MAC protocol configuration naturally propagate across all layers of the network stack, supporting realistic and repeatable experimentation.

Cooja serves as a simulation and emulation environment for Contiki-NG nodes, referred to as motes, and enables the creation of virtual wireless sensor networks without requiring physical hardware. One of the primary benefits of using Cooja is its ability to emulate realistic network behavior while providing detailed visibility into packet transmissions, routing decisions, and timing events. The simulator also supports packet capture generation in PCAP format, which allowed John and Nisha to collect traffic traces that could be analyzed offline using Wireshark. This capability was essential for evaluating performance metrics such as throughput, jitter, and inter-arrival delay in a repeatable and controlled environment.

For application-layer communication, we used the built-in RPL UDP example provided by Contiki-NG. This setup establishes a simple client-server model in which a designated server node acts as the RPL DAG root and multiple client nodes periodically transmit UDP packets containing application data. UDP was selected because it provides lightweight, connectionless communication suitable for low-power networks and allows performance behavior to be observed without additional transport-layer complexity. Using the provided example ensured consistent behavior across experiments while allowing the team to focus on the effects of topology and MAC protocol selection rather than application implementation details.

Overall, the combination of Contiki-NG and Cooja provided a flexible and realistic platform for evaluating low-power wireless networking behavior. The ability to configure different MAC protocols, define multiple network topologies, and collect detailed packet-level data directly supported the goals of this project. In addition, the platform aligns closely with Industry 4.0 and cyber-physical system concepts, where deterministic communication, reliability, and energy efficiency are critical requirements for industrial IoT and time-sensitive sensor networks.

Experiment Design:

Before we get into the design of the experiment, the goal of the experiment will be reiterated. The goal of this experiment was to evaluate the impact of different network topologies and MAC protocols on reliability and timing metrics in a simulated IoT or tactical sensor network. This experiment will help provide insights into which configurations are best suited for time-sensitive, low-power communication systems.

As mentioned in the project platform section, the experiment design included the use of Cooja within the Contiki OS platform. To perform these experiments, three topologies were chosen for study. The three that were decided on were the tree, mesh, and star topologies. The star topology was chosen due to its ease of management, scalability, and low latency[7]. The tree topology was chosen due to its efficient hierarchical scalability[7]. The mesh topology provides reliability and redundancy through multiple connection paths[7]. Once the three topologies were established. The next challenge was to recreate these topologies in the Cooja simulator. This was done by creating Cooja nodes for each of the topologies. There was one node in the experiment that acted as a server, and the remaining nodes were the client nodes. A different Cooja mote would be created for the clients and server. In order to create the motes, the user needs to click on the “Motes” dropdown menu at the top of the simulator screen, then “Add Motes”, then “Create new mote type”, then “Cooja mote”. Once the Cooja mote is selected, navigate to the rpl-udp folder and open the udp-client for the client nodes.c file. Once this is selected, you can compile the file, and the client nodes will be ready to be launched. The same process should be applied to the server node.

The team decided to use the rpl-udp network and nodes. This was chosen because the RPL (Routing Protocol for Low-power and Lossy Networks) protocol is enabled by default in most simulations that use the Contiki platform [6]. Therefore, when our simulation to collect data packets was run, the underlying routing protocol being run is RPL. Using the rpl-udp example in Cooja allowed the team to set up defined topologies while still using the preferred routing and internet protocol; therefore, the team did not have to implement these protocols themselves. Additionally, since this was a stand-alone RPL network, a border router was not set up since communication to outside devices was not needed[6]. In these examples, a direct acyclic graph (DAG) root will be set up from the udp-server.c file and the DAG nodes will be created by the udp-client.c file[6]. The client sends a UDP request periodically with a counter as payload, and once the request is received, the server sends a response with the same counter back to the originator[6]. UDP provides application-layer data transport over the RPL DAG so it enables simple client-server communication using the simple-udp implementation in Contiki[8]. The UDP implementation in Contiki is as follows: a socket is registered at both the sender and receiver. This registers the UDP connection with the host and remote port[8]. No fixed IPv6 address is given[8]. For each incoming datagram, a callback function is called such that source, destination addresses, port, and payload will be given[8]. The UDP packet size in this experiment was 16 bytes. This value was the default packet size in the rpl-udp example;

however, this value can be changed as it is defined as a macro in the source code example file[9]. The simulation duration was 12 seconds for each topology with each of the MAC protocols. Lastly, the transmission interval for each of the packets can be set through application logic. In this experiment the transmission interval for the packets was set to 200ms. However, at times based on the jitter the actual transmission interval did fluctuate anywhere from 70ms to 10s.

The topologies were created and then the simulator was started. In terms of starting the simulator this means the UDP packets began sending between the client and servers. Once this basic implementation was established of creating the topologies and sending packets it was time to experiment with the MAC protocols. The two MAC protocols that were chosen for experimentation were CSMA/CA which is the default IEEE 802.15.4 random-access scheme used in Contiki and provides flexible communication but is prone to collisions under heavy load[9]. The next MAC protocol that was chosen was TSCH (Time Slotted Channel Hopping) which is a deterministic, time-synchronized MAC layer introduced by IEEE 802.15.4e and supported in Contiki-NG[10]. This protocol offers predictable latency, reduced interference, and higher reliability — characteristics important for time-sensitive industrial IoT networks[10]. Therefore, for the experiments once the topologies were created the simulator was started and the packet capture files were generated for the CSMA/CA protocol. The simulator was run for 12 seconds for each of the topologies. In Contiki the MAC protocol can be configured in the Makefiles[9]. There is a Make variable called MAKE_MAC that can be set to the desired MAC protocol. To conduct the experiment using the TSCH MAC protocol the MAKE_MAC variable was set to MAKE_MSC_TSCH[9]. Once this is done the program files can be recompiled and the nodes for the topologies need to be recreated with the TSCH MAC protocol. So all the nodes for the three topologies were recreated for the TSCH experiment. Once this is complete a similar process was carried out as for the nodes with the CSMA protocol. The topologies were created and then the simulator was run for 12 seconds for each of the topologies and packet capture files were taken.

The performance metrics the experiment aimed to collect and analyze were throughput, jitter, latency (inter-arrival delay) and packet loss. In order to collect these metrics the team captured the packet data from the Cooja simulation. This was done from the radio messages tool in the Cooja simulation. This tool allowed the 6LoWPAN Analyzer with PCAP option to be toggled on. From this tool packet capture files were generated and put in the same folder as the Cooja build. Once these files were created they were analyzed for the given performance metrics using Wireshark. The detailed analysis of the experiment results from these performance metrics will be detailed in the “Experiment Results” section. A different packet capture file was generated for each topology with the two different MAC protocols. The jitter and throughput metrics were easily calculated and analyzed through Wireshark. However, for the latency metric this was slightly difficult to calculate from the packet captures since there were no timestamps natively in the code. The team was able to add timestamps to the packets by modifying the udp-server.c and udp-client.c files. However, these timestamps did not completely serve their purpose because it was still not able to determine the time the packets took to go from sender to

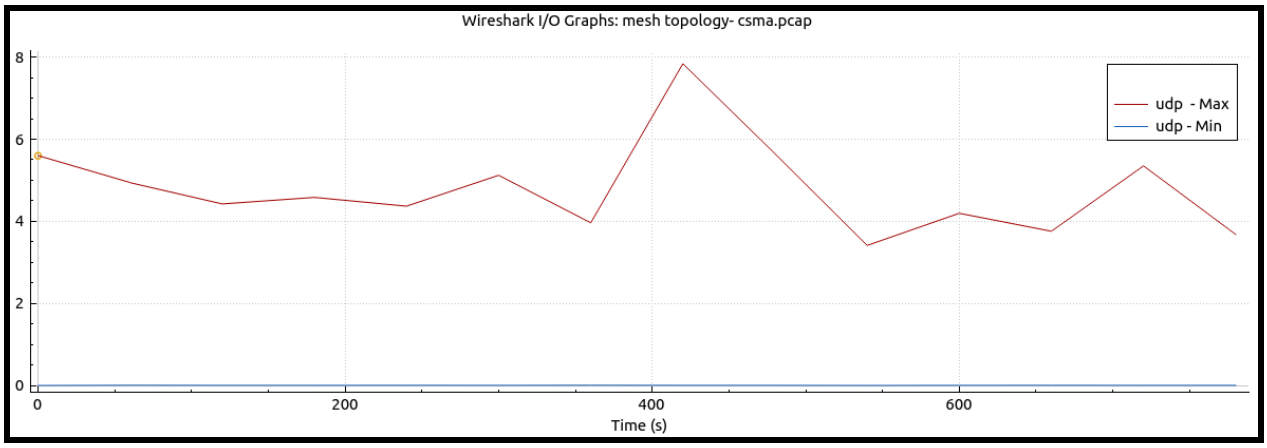
receiver. The team also explored adding an ICMP ping to the packet captures but were unable to do so unless a border router was set up to an outside network. Since this was not the goal of the experiment the team did not go down this route. Instead the team decided to use inter-arrival delay since the delay from packet being sent to the time they are received was able to be calculated. Therefore, in the results delay will be substituted with interarrival delay. Lastly, when measuring for packet loss the team noticed there was no point in the traffic where drop offs in packets occurred. Meaning from the results packet loss was not observed such that the packets received ever equaled zero. Therefore, after analyzing the packet loss measuring tools and the experiment design the team realized due to the simulation environment there was nothing in the experiment causing packet loss between the nodes. The congestion control at the routers was clearly defined and the sender was not bogging down the network by overwhelming the receiver or the network. Therefore, not creating any packet loss from congestion. The wireless links due to such an enclosed simulation environment also did not have any issues that could have caused packet losses. Therefore, the team concluded that seeing no packet loss in this environment was not unusual. However, the team does note even further testing should be conducted to further confirm these results.

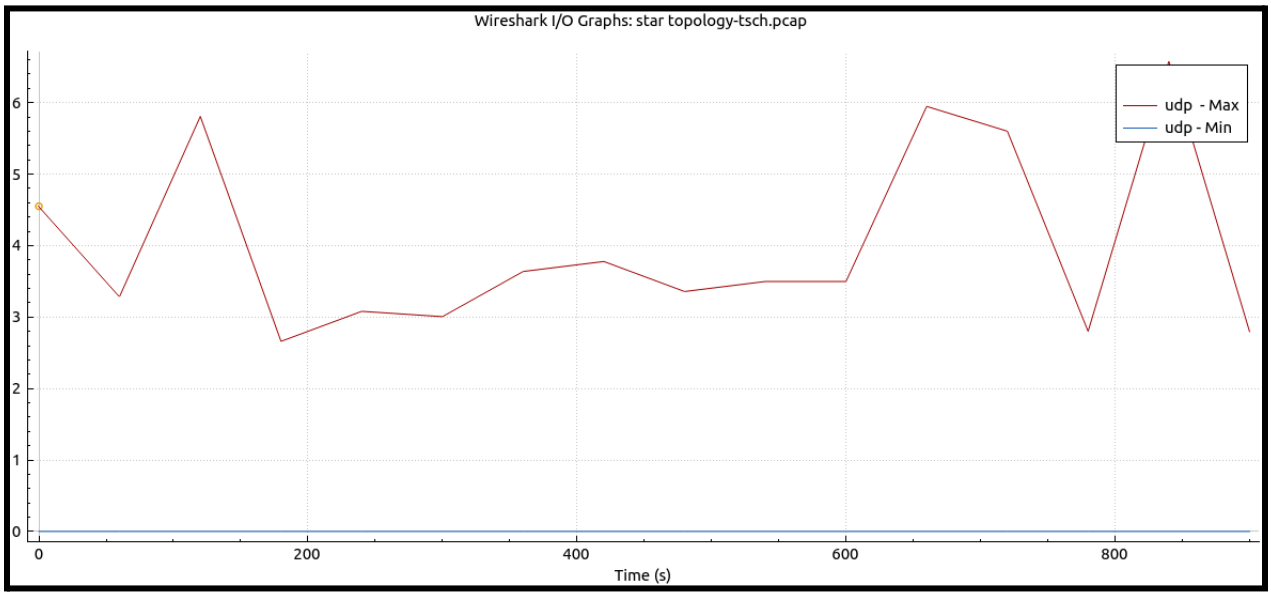
Experiment Results:

The recorded information from our experiment was logged in a packet capture (PCAP) and parsed by the packet analysis tool Wireshark. Using graphing capabilities provided within Wireshark we generate the following 4 graphs for all network topologies and MAC protocol combinations: jitter graphs which show the minimum and maximum arrival times for UDP and 802.14.5 traffic, throughput graphs, which highlight the overall packet transmission during the observation period, and an inter-arrival delay graph which plots the average delay for a packet to reach its intended destination. In the analysis of the experiment discussed below, the graphs for each topology and MAC protocol will be listed after the experimental conclusions have been discussed.

Based on our experimental analysis, we have concluded that Tree TSCH offers the best overall performance in terms of MAC protocol and network topology. Before expanding on why this combination of topology and MAC protocol is the most optimal for our use case, it is important to first examine the provided data for the given MAC protocols and topologies.

Examining the MAC protocols used, we can begin to observe how TSCH offers better performance over CSMA/CA by first analyzing the observed network Jitter in both UDP and 802.14.5 traffic across all topologies. Most notably in the network topologies where CSMA is utilized, there are noticeably larger averages in traffic delay in the perceived maximum delay times. The figures below illustrate how these jitter differences can be observed when comparing the UDP traffic jitter between TSCH and CSMA/CA in a star topology.

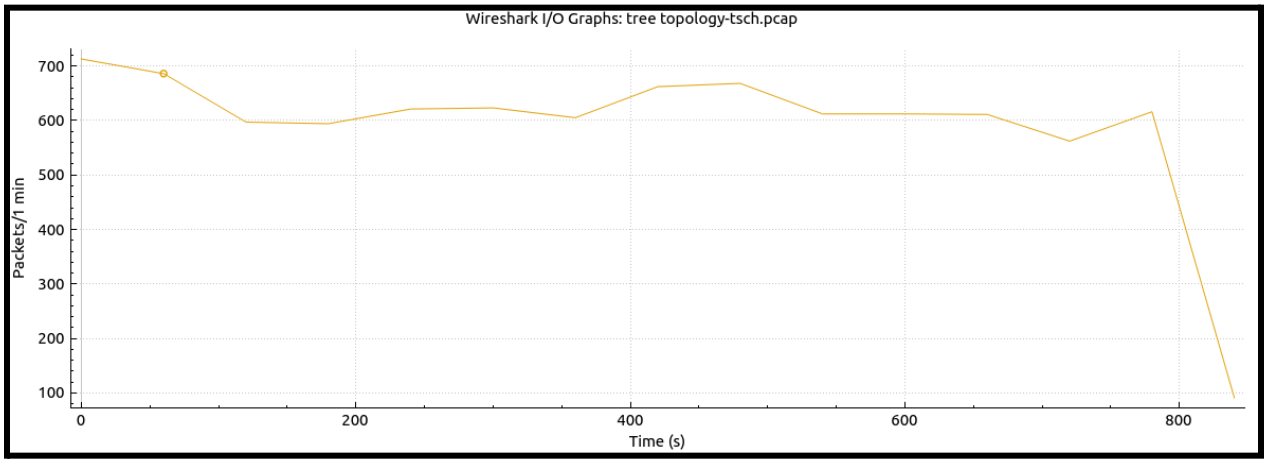


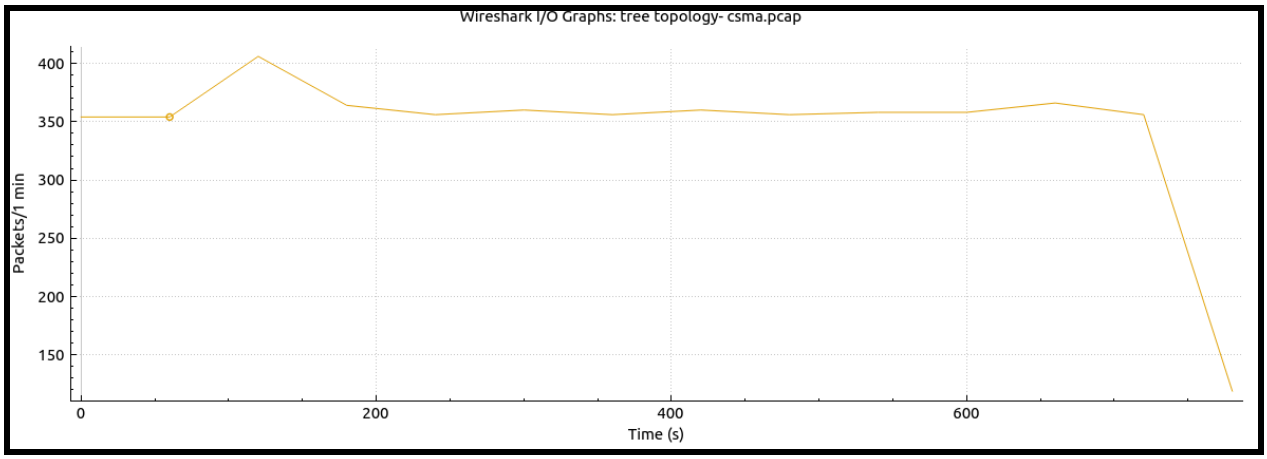


From observing the graphs shown above, we can clearly see that there is a slightly larger range in record values for the CSMA/CA MAC protocol implementation compared to the TSCH plot. The reduced jitter within TSCH MAC implementations is logical, as TSCH provides fixed transmission time slots, unlike CSMA/CA. With fixed transmission timings, there is less variation in the arrival of packets. The general trend of TSCH outperforming CSMA/CA is observed across all performance metrics in this experiment. Some of these observed benefits can be attributed to the fact that TSCH does not utilize random backoff periods. The utilization of random backoff would heavily contribute to high variance in traffic behavior and would not be conducive to creating behavior that is equally deterministic to the behavior observed in TSCH traffic. Since our packet timing is bounded by clock synchronization and not by random back-offs resulting from collisions, our TSCH networks perform better on average than the CSMA/CA implementations overall.

Given the beneficial performance of TSCH as a MAC protocol, we recognized that a corresponding topology was needed to achieve the best performance metrics across the board. With a star topology, the network is capable of effective management through a single point of failure, which can help reduce data collisions due to direct transmission from network nodes to the hub [11]. Mesh networks should enhance network reliability and minimize the number of coverage gaps within a network. Dynamic route allocation through different nodes may also provide more efficient routing and higher throughput on the network [12]. Lastly, with tree topologies, the expected performance benefits of a hybrid between star and bus topologies are realized. Tree topologies can provide more robust networking solutions capable of sustaining more errors and or network faults. Based on our experimentation and analysis of the three listed topologies, our research indicates that the tree topology is the most suitable topology for routing in our use case. The conclusion that a tree topology would be the most optimal was drawn from the high throughput, low jitter, and minimal inter-arrival delay observed in the PCAP graphs, not only for a tree topology using TSCH but also for CSMA/CA.

Our conclusion contradicts initial assumptions made by the team that a star topology would provide better performance, since a star topology is better suited for handling collisions. One clear indicator of tree topology TSCH being the best suited for our application was the observed throughput. Examining the graphs below, we can observe that the tree topology TSCH implementation has noticeably higher throughput.



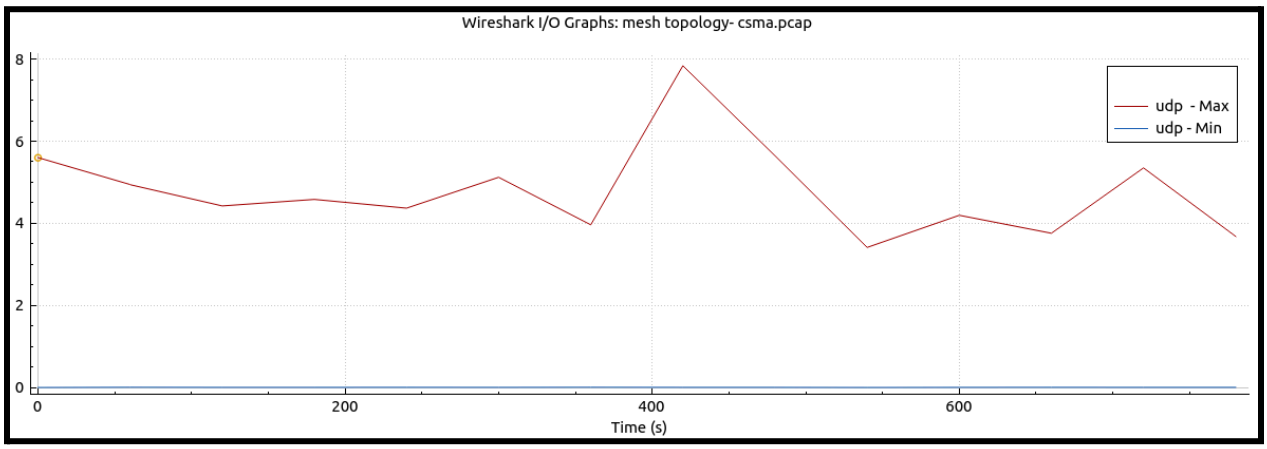


Tree topology TSCH had roughly double the amount of packets per minute that we observed in tree topology CSMA/CA. While throughput is a great indicator of the topology's performance, we compared the other data plots of UDP and 802.14.5 jitter and inter-arrival delay. Examining the jitter data plots, in particular, reveals that tree topology consistently produces lower average maximum traffic delay times and minimal inter-arrival delay times.

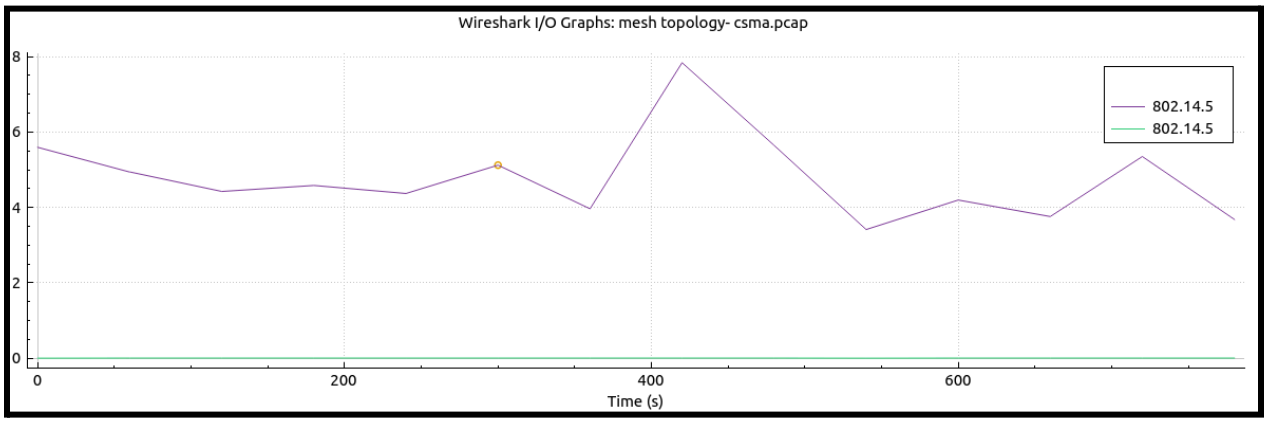
For additional information on the data collected, please refer to the individualized graphs below of all network topologies and MAC protocols:

Mesh Topology CSMA

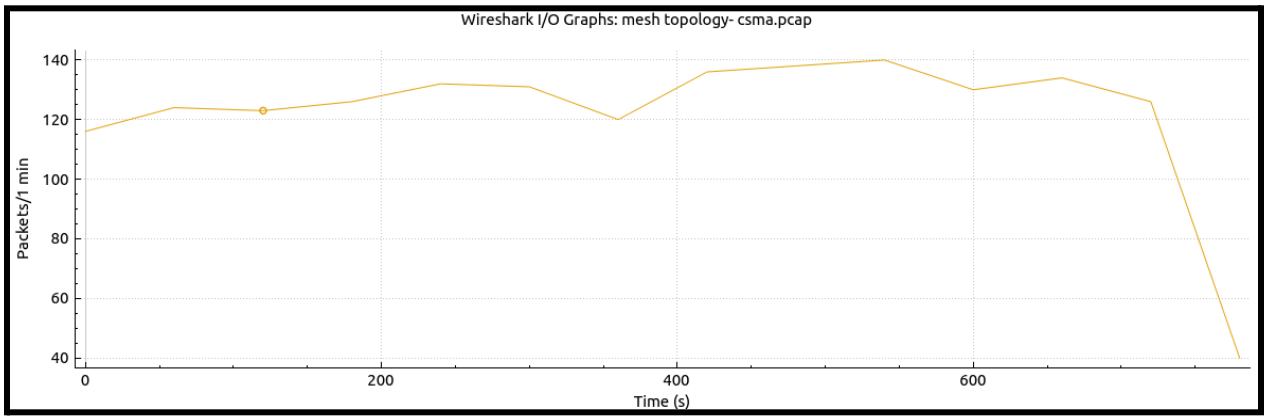
UDP Jitter:



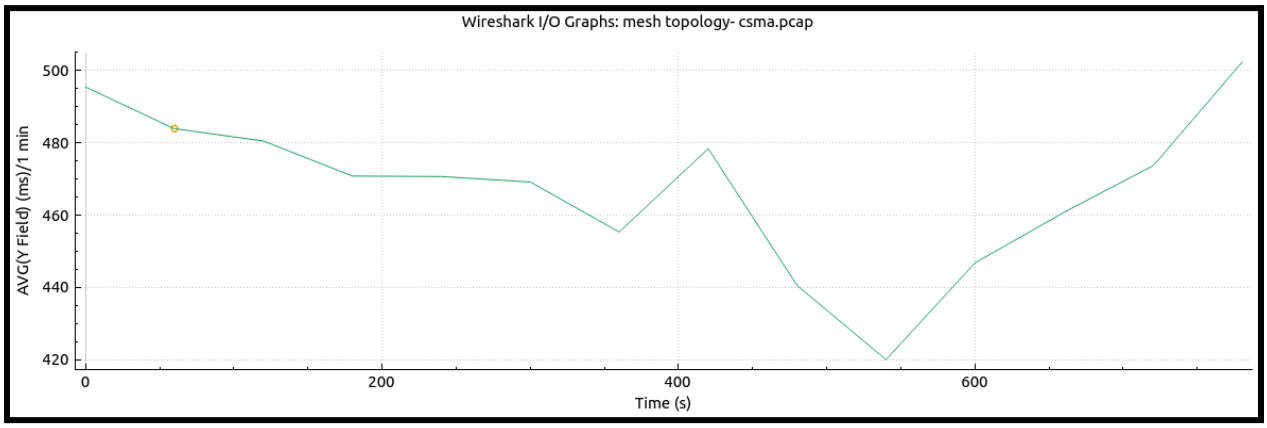
802.14.5 Jitter:



Throughput:

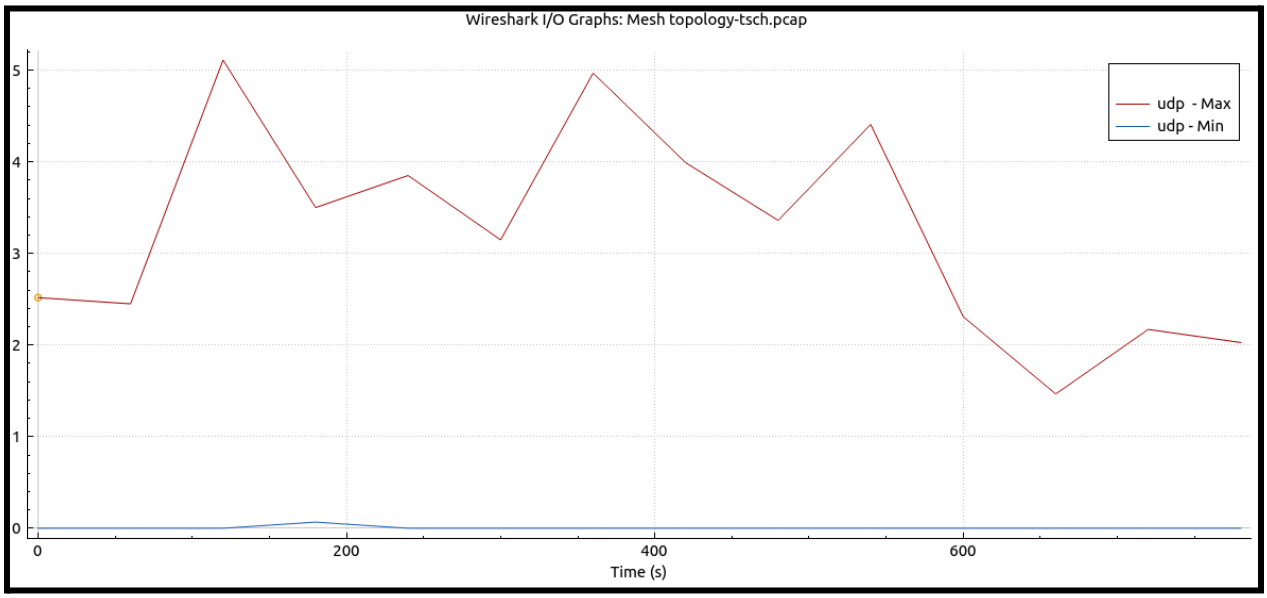


Inter-Arrival Delay:

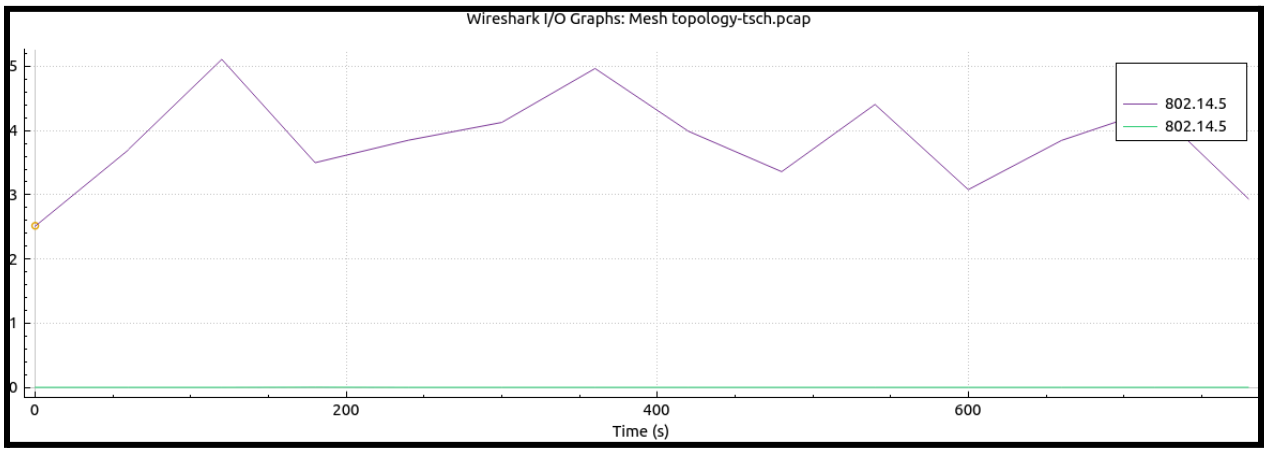


Mesh Topology TSCH

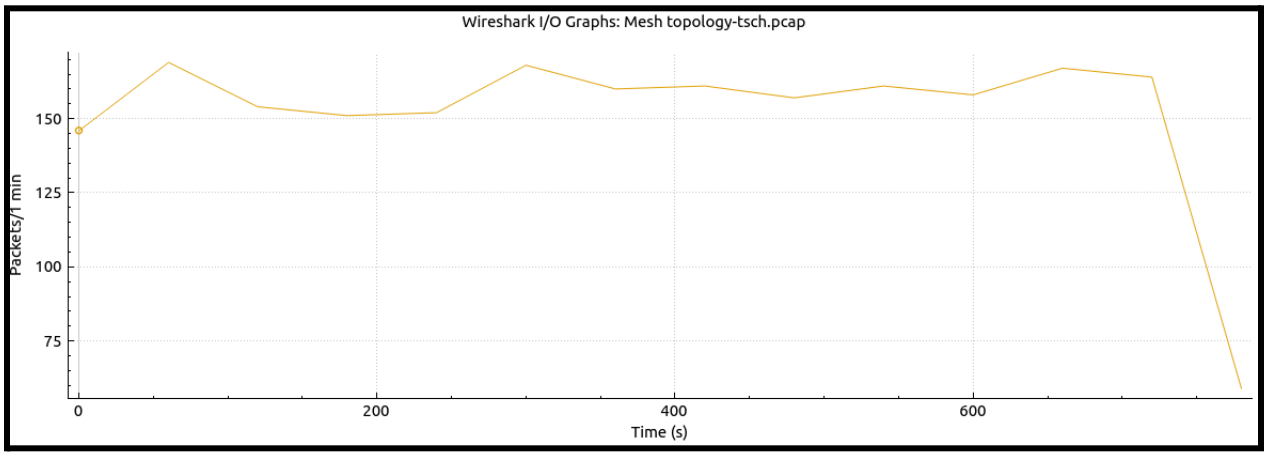
UDP Jitter:



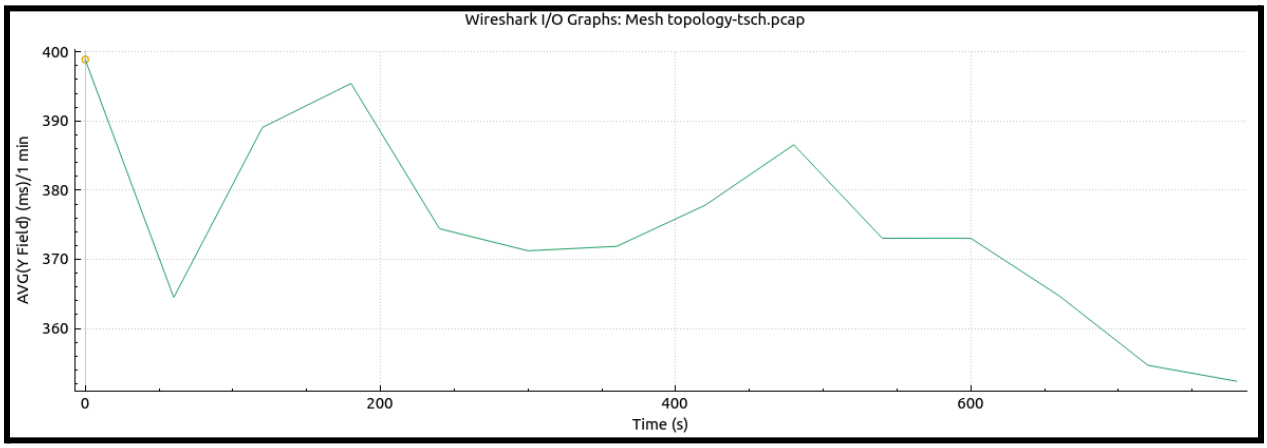
802.14.5 Jitter:



Throughput:

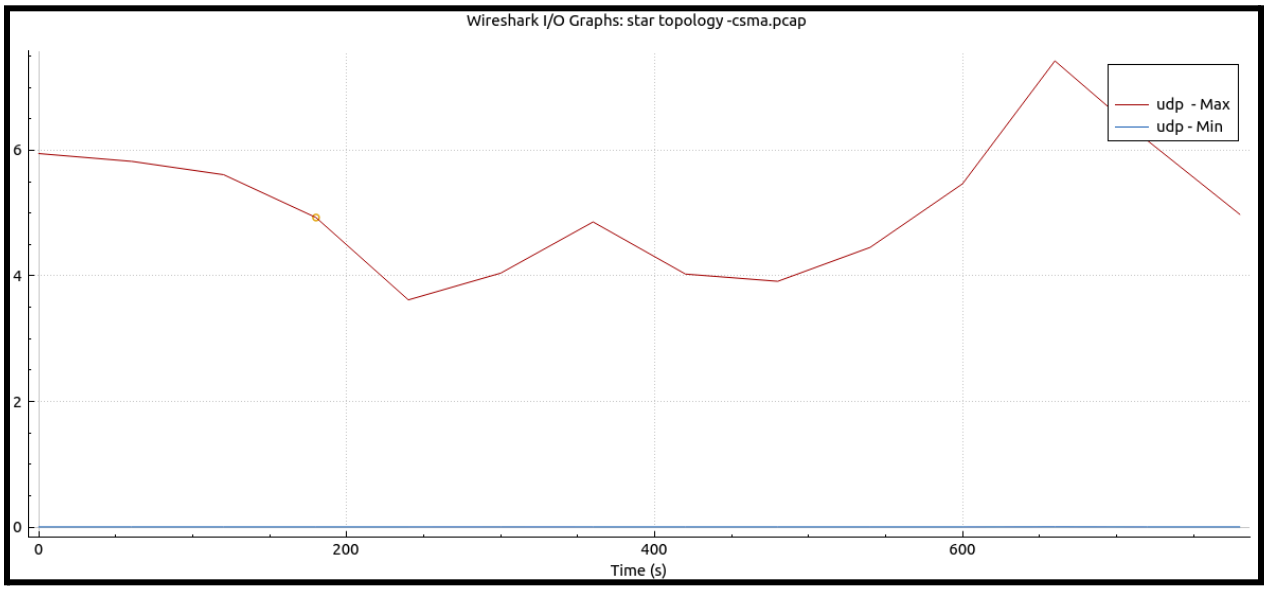


Inter-Arrival Delay:

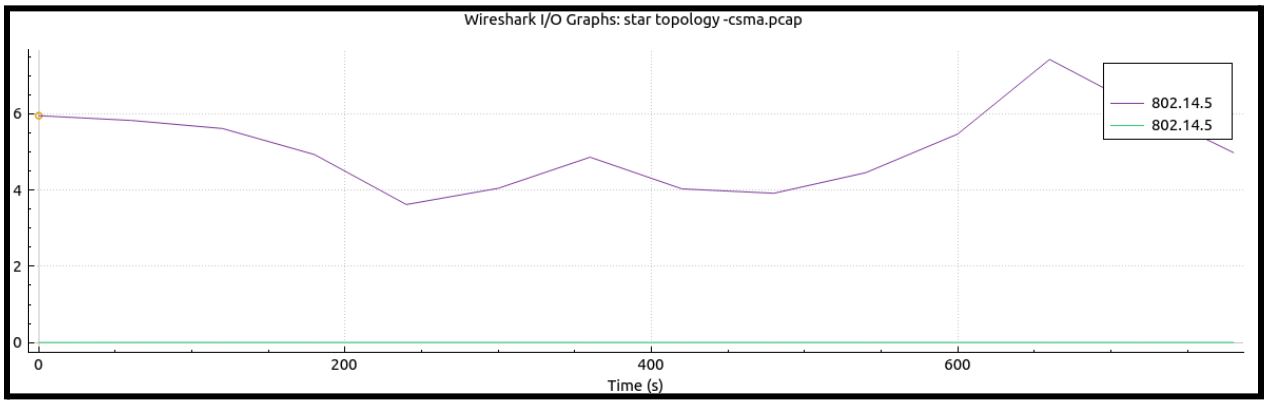


Star Topology CSMA

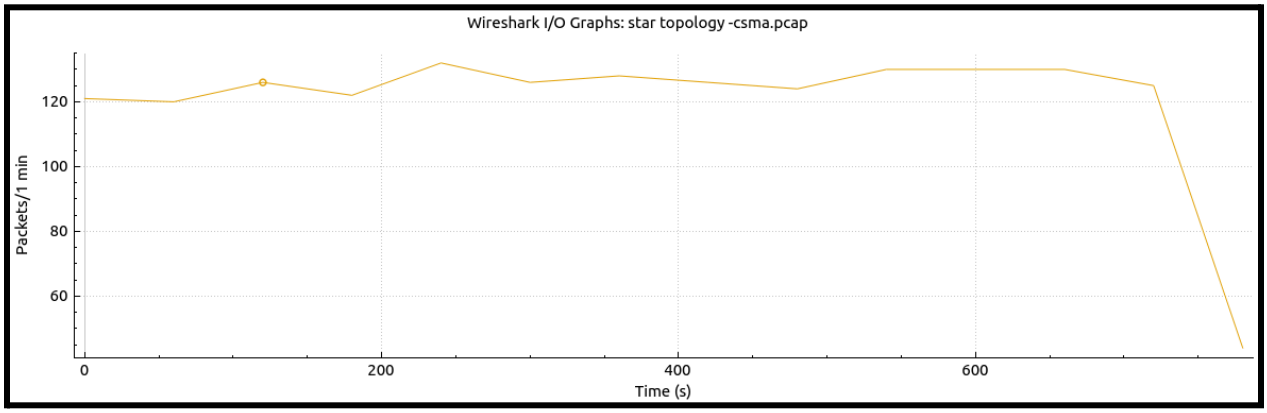
UDP Jitter:



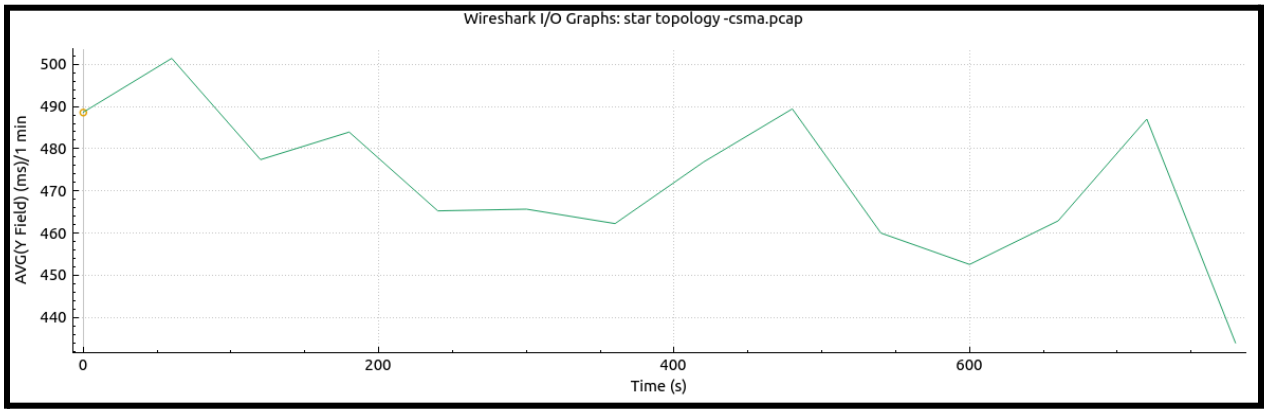
802.14.5 Jitter:



Throughput:

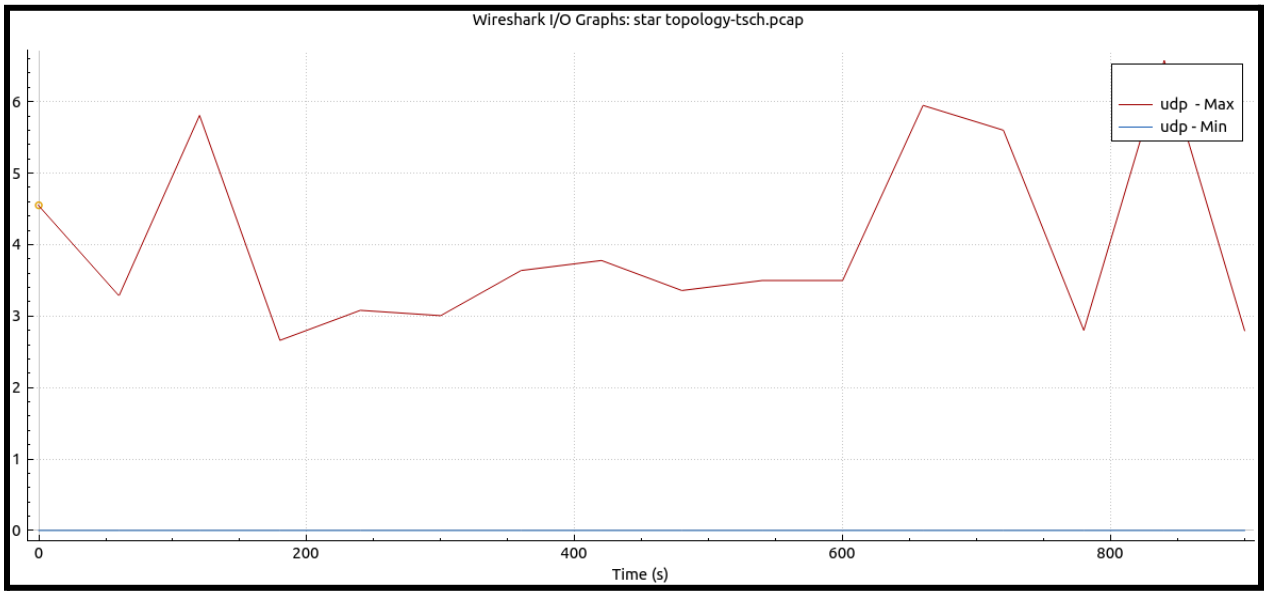


Inter-Arrival Delay:

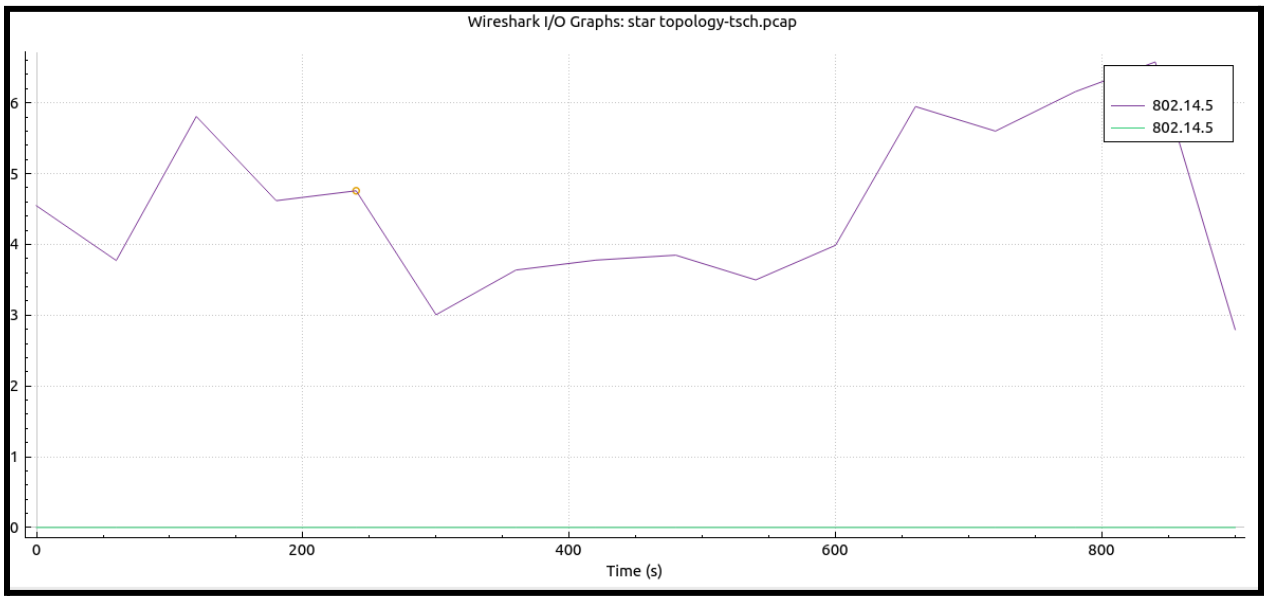


Star Topology TSCH

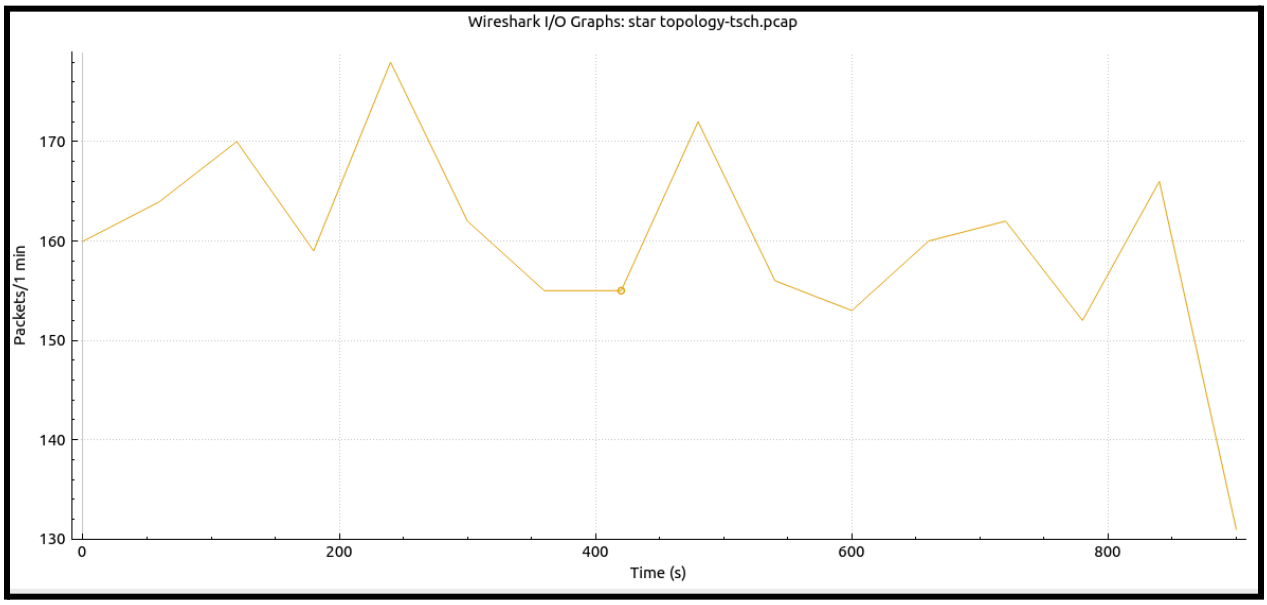
UDP Jitter:



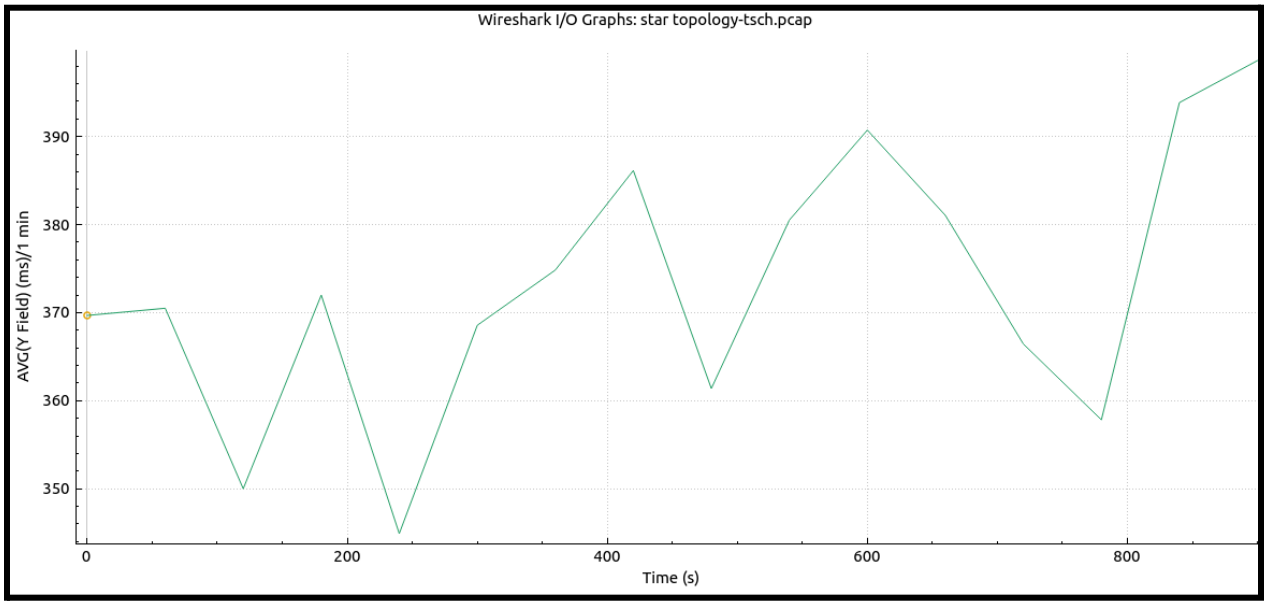
802.14.5 Jitter:



Throughput:

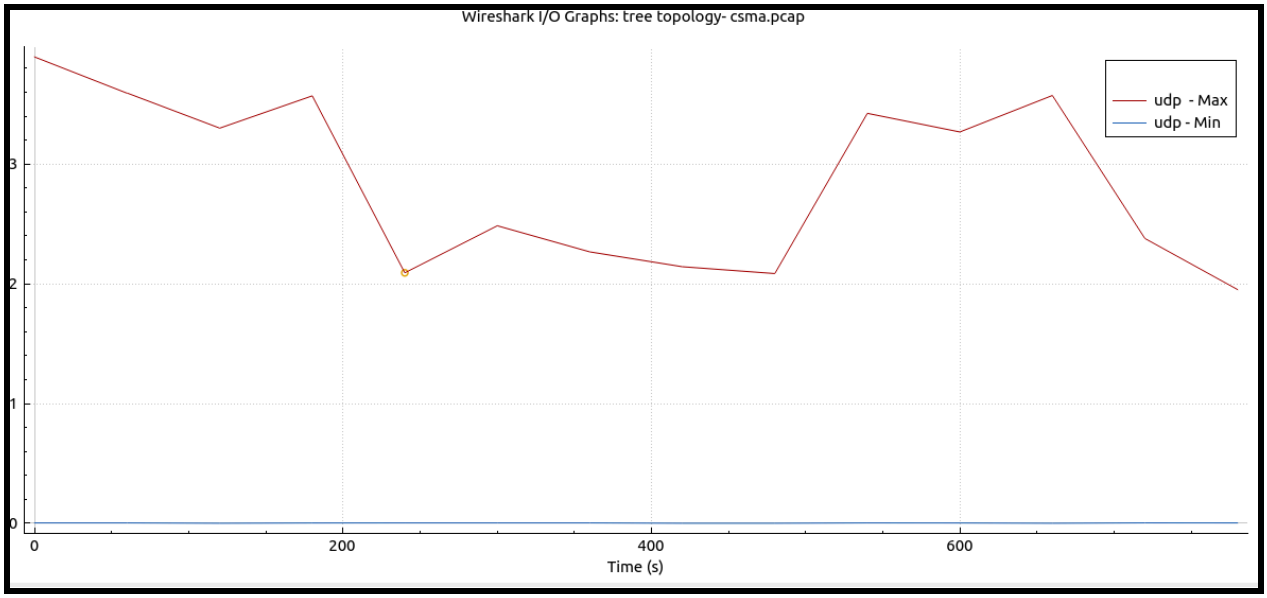


Inter-Arrival Delay:

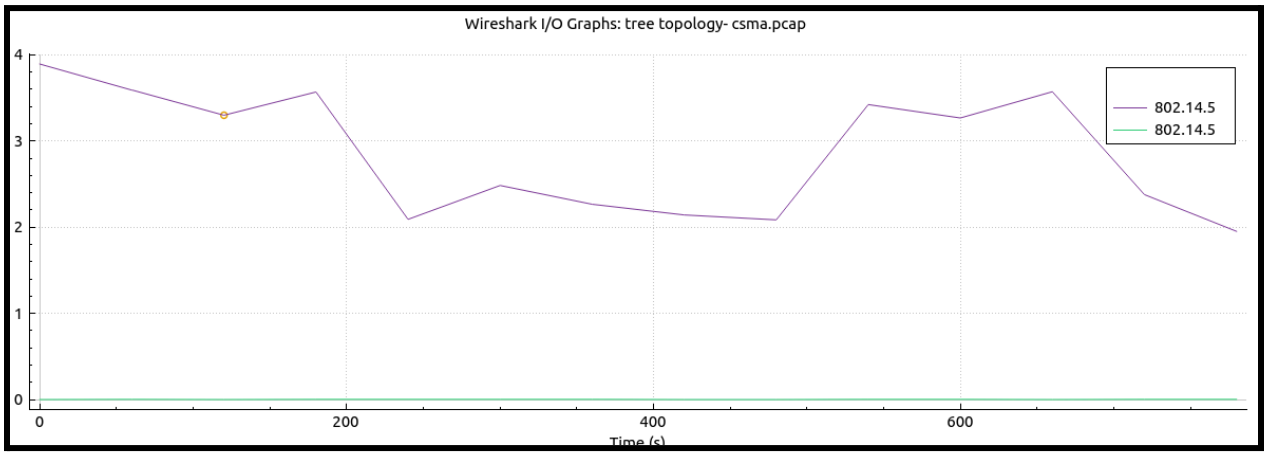


Tree Topology CSMA

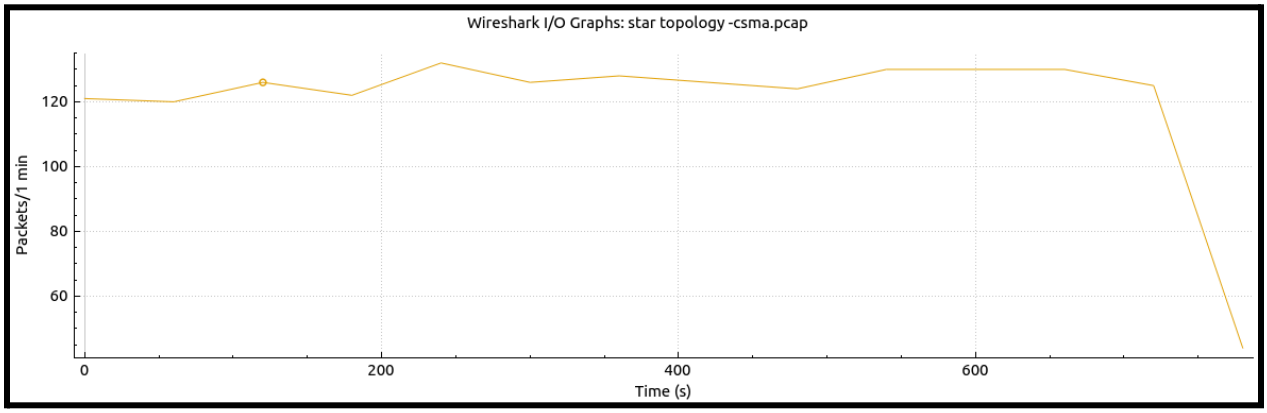
UDP Jitter:



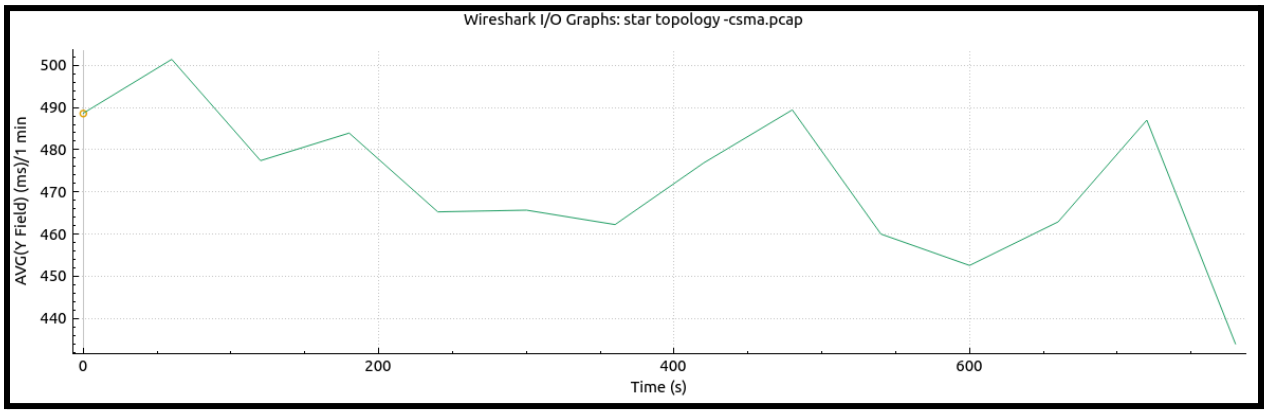
802.14.5 Jitter:



Throughput:

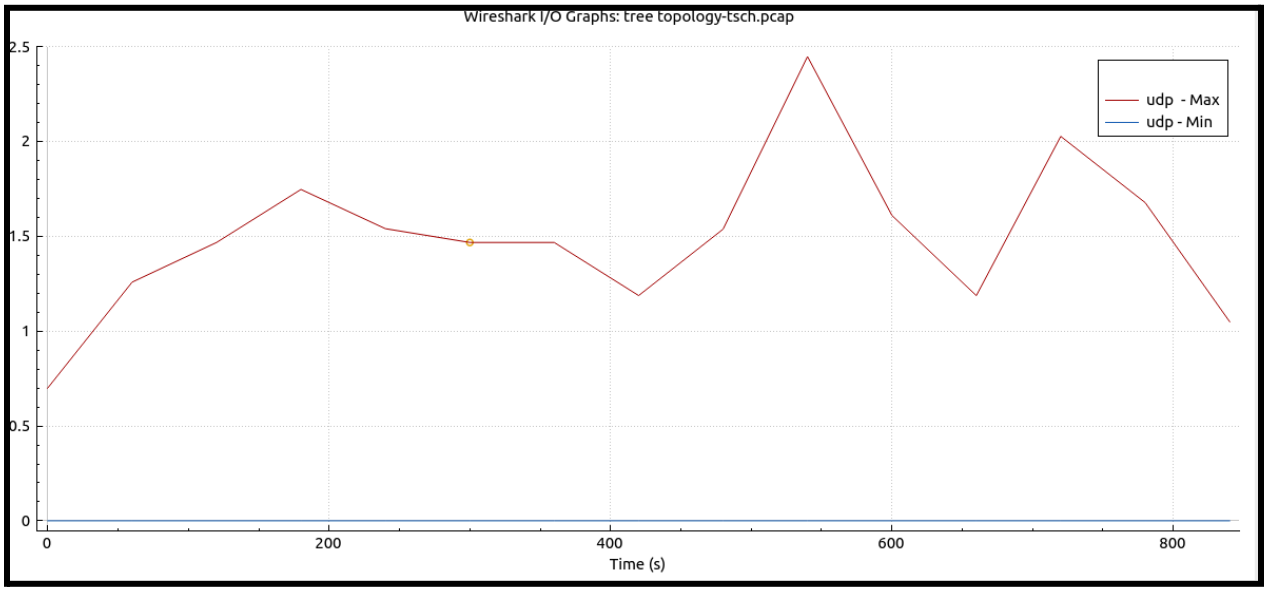


Inter-Arrival Delay:

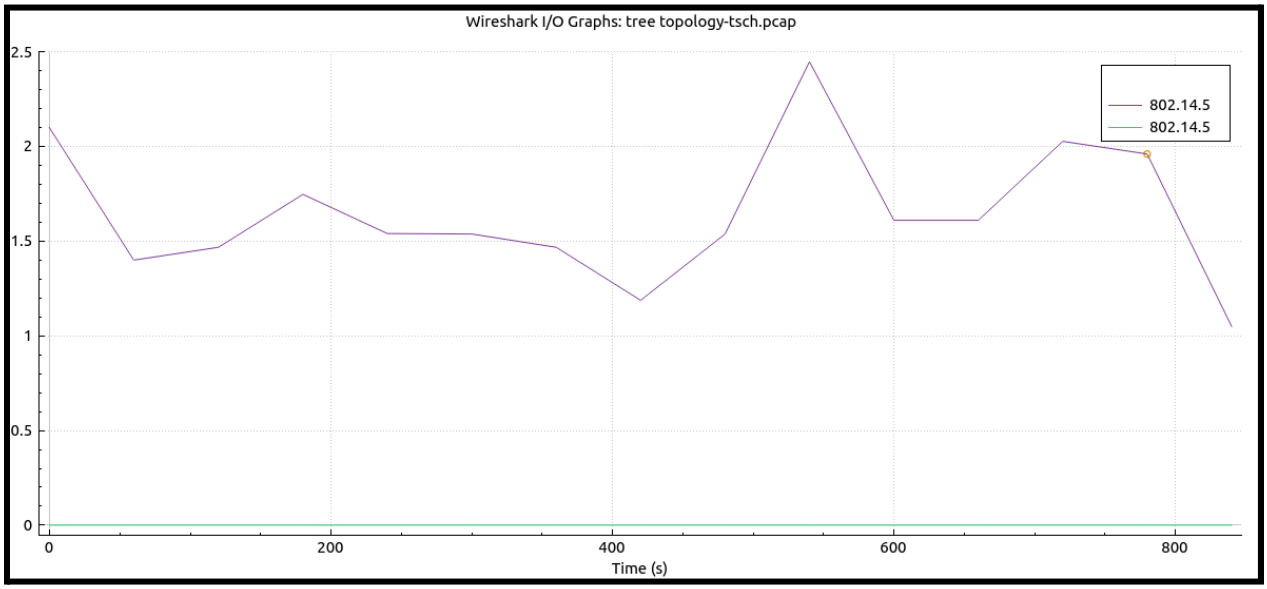


Tree Topology TSCH

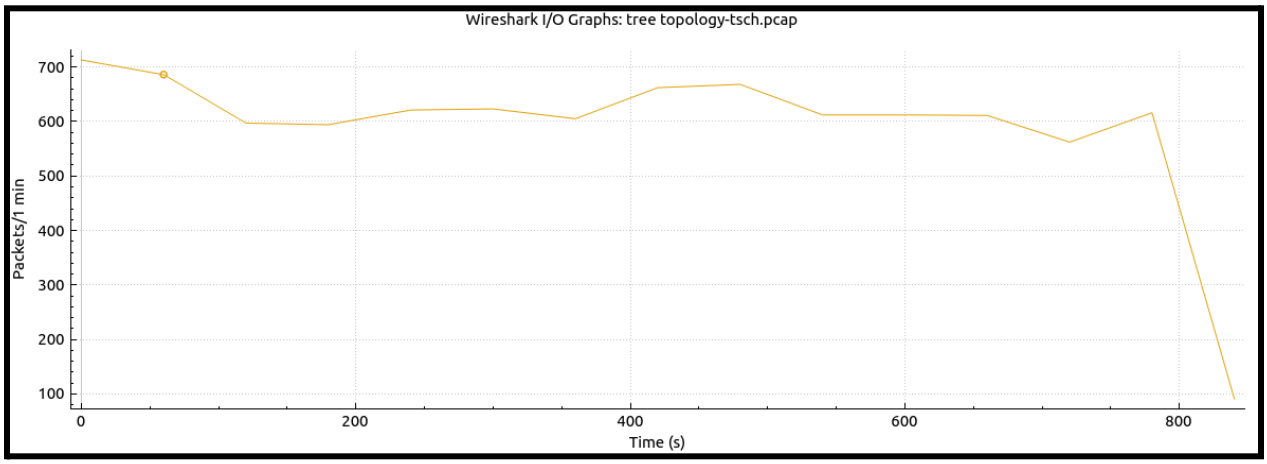
UDP Jitter:



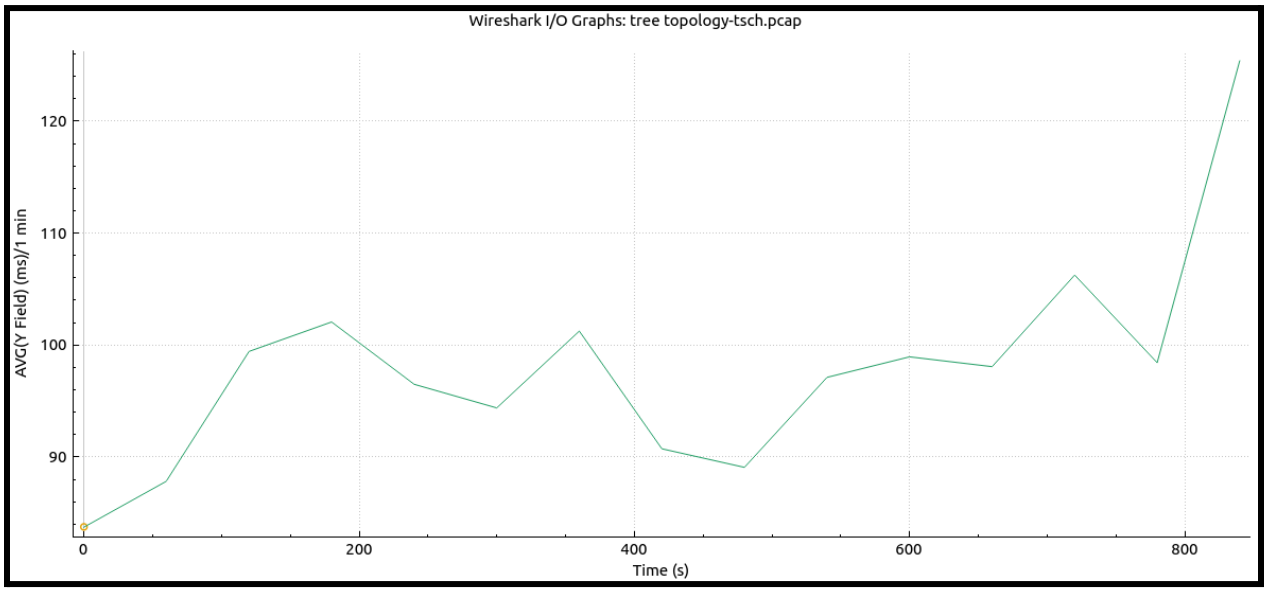
802.14.5 Jitter:



Throughput:



Inter-Arrival Delay:



Conclusion:

In this project, Team One investigated the impact of various MAC protocols and network topologies on reliability and timing performance in a simulated low-power IoT and tactical sensor network. Using the Contiki OS and the Cooja simulator, we compared CSMA/CA and TSCH across star, mesh, and tree topologies. Performance was evaluated using jitter, throughput, and inter-arrival delay metrics collected from Wireshark packet captures. Across all test cases, we observed that TSCH consistently outperformed CSMA/CA, particularly for metrics related to time-sensitive communication.

For every topology tested by John and Nisha, TSCH produced lower and more consistent jitter in both UDP and IEEE 802.15.4 traffic when compared to CSMA/CA. When looking specifically at topology performance, the tree topology consistently performed better than both star and mesh configurations under both MAC protocols. In our results, the tree topology showed higher throughput, lower jitter, and shorter inter-arrival delays overall.

Going into the project, we expected the star topology to perform best due to its centralized structure and reduced likelihood of collisions. However, the many graphs and data points collected did not support that assumption. Instead, the hierarchical structure of the tree topology seemed to provide a better balance between efficiency, scalability, and reliability.

Mesh topologies, while more resilient due to path redundancy, introduced additional routing overhead in our tests, which negatively impacted the metrics we measured. In contrast, for a low-power, time-synchronized, and mostly static sensor network, the combination of TSCH and a tree topology resulted in more predictable and reliable performance. These results reinforce the idea that topology selection should be driven by MAC layer behavior and implementation requirements rather than assuming a single topology is best suited in all scenarios.

There are also important limitations to acknowledge. Each simulation was run for approximately 12 seconds of simulation time by Nisha, which corresponds to roughly 800 seconds, or about 13 minutes, when scaled to real-world low-power network operation (This is what time scale you see in the graphs and charts). Even with this limitation, the performance trends were consistent across all measured metrics. Based on these results, a TSCH based MAC protocol combined with a tree topology appears to offer a reliable and deterministic solution for time-sensitive IoT and tactical communication environments and aligns well with current Time Sensitive Networking and Industry 4.0 concepts.

Looking ahead, these findings raise several opportunities for future research. Additional MAC protocols, routing strategies, or hybrid topologies could be explored to better understand how different design choices perform under varying conditions. Repeating similar experiments using other simulation tools or testbeds would also help determine whether the observed

performance advantages are inherent to the protocols and topologies themselves or influenced by assumptions within the Contiki and Cooja environment or by limitations in our project's design and execution. Expanding the scope in this way would strengthen confidence in the results and provide a clearer picture of how MAC and topology decisions translate to real-world performance.

Team Member Contributions:

Captain Russell (Team Lead): Responsible for team coordination, setting milestones, and overall report leadership. Will draft the Project Platform overview to set the stage for the specific experimentation and design used in this report. Will also contribute to the project's conclusions and ensure alignment with Industry 4.0 communication standards.

John: Researched Contiki and Cooja platforms and successfully installed and configured the toolchain on a Windows environment. Collected and shared setup resources with teammates. Identified areas of overlap between time-sensitive networking (TSN) concepts and Contiki's MAC protocols. Utilized Contiki/Cooja to generate packet captures from network traffic in Cooja. Will lead experiments comparing network topologies within Cooja and collect metric data using Cooja logs and Wireshark.

Nisha: Researched and experimented with multiple different network topologies. Collected and shared resources about running TSCH and CSMA/CA in the Cooja environment. Led the efforts on experimenting and running the TSCH example file and collecting baseline metrics. Researched methods to measure packet loss, latency, jitter, and throughput, and Wireshark integration for packet analysis. Will continue researching and experimenting with executing MAC protocols over the nodes.

References:

1. M. Ashjaei, L. Lo Bello, M. Daneshtalab, G. Patti, S. Saponara, and S. Mubeen, "Time-Sensitive Networking in automotive embedded systems: State of the art and research opportunities," *Journal of Systems Architecture*, vol. 117, p. 102137, Aug. 2021, doi: https://doi.org/10.1016/j.sysarc.2021.102137.

2. J. John, Md. Noor-A-Rahim, A. Vijayan, H. V. Poor, and D. Pesch, "Industry 4.0 and Beyond: The Role of 5G, WiFi 7, and Time-Sensitive Networking (TSN) in Enabling Smart Manufacturing," Future Internet, vol. 16, no. 9, p. 345, Sep. 2024, doi: https://doi.org/10.3390/fi16090345.

3. C. San, J. Lander, I. Z. Ugarte, and V. M. Vilches, "Time-Sensitive Networking for robotics," *arXiv.org*, 2018. https://arxiv.org/abs/1804.07643

4. F. Zhang, M. Ren, and B. Zhang, "Design and Implementation of MAC with Time-Sensitive Networking," *2022 3rd International Conference on Computer Vision, Image and Deep Learning & International Conference on Computer Engineering and Applications (CVIDL & ICCEA)*, pp. 502–506, May 2022, doi: https://doi.org/10.1109/cvidliccea56201.2022.9824852.

5. T. Zhang, G. Wang, C. Xue, J. Wang, M. Nixon, and S. Han, "Time-Sensitive Networking (TSN) for Industrial Automation: Current Advances and Future Directions," *ACM Computing Surveys*, Sep. 2024, doi: https://doi.org/10.1145/3695248.

6. "rpl-udp — Contiki-NG documentation," *Contiki-ng.org*, 2017. https://docs.contiki-ng.org/en/develop/examples/rpl-udp/README.html

7. GeeksforGeeks, "Types of Network Topology," *GeeksforGeeks*, Mar. 07, 2017. https://www.geeksforgeeks.org/computer-networks/types-of-network-topology/

8. "UDP communication — Contiki-NG documentation," *Contiki-ng.org*, 2017. https://docs.contiki-ng.org/en/master/doc/programming/UDP-communication.html

9. "The Contiki‑NG configuration system — Contiki-NG documentation," *Contiki-ng.org*, 2017. https://docs.contiki-ng.org/en/master/doc/getting-started/The-Contiki-NG-configuration-system.html

10. "RPL — Contiki-NG documentation," *Contiki-ng.org*, 2017. https://docs.contiki-ng.org/en/develop/doc/programming/RPL.html
11. "Advantages and Benefits of a Star Network Topology," Stellastra, Jan. 2025. https://stellastra.com/advantages-benefits-of-star-network-topology/
12. "Mesh Networking Systems: Coverage, Performance, and Setup Ease - serverpipeline.com," *WordPress*, Jul. 28, 2025. https://serverpipeline.com/mesh-networking-systems-coverage-performance-and-setup-ease/#what-are-the-benefits-of-using-mesh-networking-systems (accessed Dec. 18, 2025).